

# **Assignment 7**

## **Secure Network Application Development Using TCP**

*Secure Multi-Client GUI Chat Application*

Name: Madhab Boro

Roll No.: CSB24055

Course: Computer Networks Laboratory

*Report Submitted as Part of Assignment 7 Evaluation*

## 1. Objective

---

The objective of this assignment is to enhance the GUI-based multi-client TCP chat application developed in Assignment 6 by incorporating practical application-layer security mechanisms. This includes user authentication, secure password storage using SHA-256 hashing, prevention of duplicate logins, input validation, protection against repeated failed login attempts, session management with inactivity timeout, and secure event logging. The updated application is verified using Wireshark to confirm that authentication and communication occur correctly over TCP port 5000, and that no sensitive information such as plaintext passwords is exposed on the network or in log files.

The emphasis of this assignment is on practical application security rather than advanced cryptographic techniques, reinforcing key concepts of authentication, authorization, and secure session handling in a client-server networking environment.

## 2. Security Features Implemented

---

### 2.1 User Authentication

The application requires every client to provide a valid username and password before being granted access to the chat system. Credentials submitted by the client are verified against a locally maintained user database (users.json) on the server side. Only successfully authenticated users are allowed to send or receive messages.

### 2.2 Secure Password Storage

Passwords are never stored or transmitted in plaintext. Instead, each password is hashed using the SHA-256 algorithm (via Python's `hashlib.sha256()`) before being stored in `users.json`. During login, the password entered by the client is hashed and compared against the stored hash, ensuring that the original password is never persisted on disk.

```
# Example: password hashing (server-side)
import hashlib

def hash_password(password: str) -> str:
    return hashlib.sha256(password.encode()).hexdigest()
```

### 2.3 Duplicate Login Prevention

The server maintains an in-memory record of currently logged-in usernames. If a login request is received for a username that is already active in another session, the server rejects the request and notifies the client that the account is already logged in elsewhere. This prevents session hijacking through simultaneous logins and ensures a one-to-one mapping between a user and an active session.

### 2.4 Input Validation

All client input is validated on the server before being processed, including:

- Usernames are checked against a defined format (alphanumeric, restricted length) and empty usernames are rejected.
- Empty or whitespace-only passwords are rejected at login.

- Message size is capped, and oversized messages are discarded with a warning sent to the client.
- Unsupported or malformed commands are rejected instead of being forwarded to other clients.

This prevents malformed or malicious input from disrupting server operation or affecting other connected clients.

## 2.5 Failed Login Protection

The server tracks the number of consecutive failed login attempts per username. After five consecutive failures, further login attempts for that account are temporarily blocked for a defined cooldown period. This mitigates brute-force password guessing attacks against the authentication mechanism.

## 2.6 Session Management and Secure Logging

Each authenticated session is tracked using a last-activity timestamp. If a client remains inactive beyond a configured timeout period, the server automatically logs the client out and notifies connected users. Clients may also log out manually using the Disconnect button or the `/logout` command. All security-relevant events — successful logins, failed login attempts, duplicate login attempts, account lockouts, timeouts, and logouts — are recorded in `security_log.txt` with timestamps. Passwords and other sensitive data are never written to the log file.

# 3. System Architecture

---

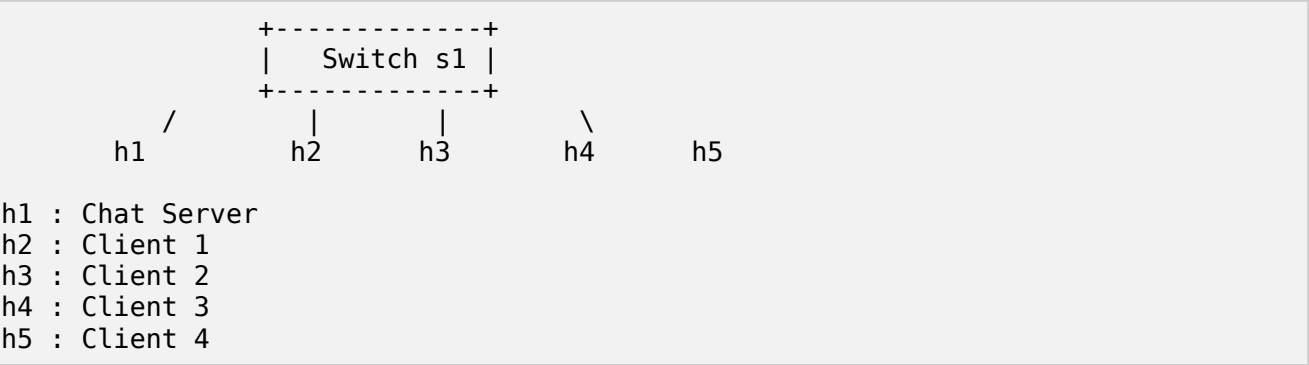
The application follows a client-server architecture built on Python socket programming with multithreading, extended from the Assignment 6 implementation. Networking logic is kept independent of the GUI layer so that authentication, session, and message-handling logic can be modified without affecting the Tkinter-based interface.

## 3.1 Component Overview

| Component                            | Description                                                                                                         |
|--------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| <code>server.py</code>               | Multithreaded TCP server; handles authentication, session tracking, input validation, message routing, and logging. |
| <code>client_gui.py</code>           | Tkinter-based GUI client; handles login screen, chat window, and communicates with the server over TCP.             |
| <code>users.json</code>              | Stores usernames and SHA-256 password hashes.                                                                       |
| <code>security_log.txt</code>        | Stores timestamped security events (logins, failures, lockouts, timeouts, logouts).                                 |
| <code>chat_history.csv</code>        | Stores a record of exchanged chat messages.                                                                         |
| <code>performance_results.csv</code> | Stores performance/latency statistics collected during testing.                                                     |

### 3.2 Network Topology

The application was tested using a Mininet single-switch topology with five hosts, created using the command `sudo mn --topo single,5`. One host acts as the chat server, while the remaining four act as independent GUI clients connected through a common virtual switch.



Connectivity between all hosts was verified using the `pingall` command prior to running the application, confirming that the virtual network was correctly established before testing the TCP-based chat application.

## 4. Implementation

### 4.1 Authentication Flow

1. The client GUI collects a username and password from the user and sends a login request to the server over TCP.
2. The server checks whether the account is currently locked due to repeated failed attempts; if locked, the request is rejected.
3. The server checks whether the username is already logged in from another session; if so, the login is rejected as a duplicate.
4. The submitted password is hashed using SHA-256 and compared against the stored hash in `users.json`.
5. On success, the session is registered in memory with a timestamp, a success event is logged, and the client is granted access to the chat.
6. On failure, the failed-attempt counter for that username is incremented, the event is logged, and an appropriate error message is returned to the client.

### 4.2 Message Handling and Input Validation

Once authenticated, every message sent by a client passes through a validation layer on the server before being processed. Messages exceeding the configured maximum length are rejected. Commands are parsed and matched against a whitelist of supported operations (`broadcast`, `/msg`, `/list`, `/logout`); anything outside this set is rejected without being forwarded.

| Command / Input                                | Behaviour                                              |
|------------------------------------------------|--------------------------------------------------------|
| Plain text message                             | Broadcast to all connected, authenticated clients.     |
| <code>/msg &lt;user&gt; &lt;message&gt;</code> | Delivered privately to the specified online user only. |

|                      |                                                                                      |
|----------------------|--------------------------------------------------------------------------------------|
| /list                | Returns the list of currently online, authenticated users.                           |
| /logout              | Ends the session, removes the user from the active session list, and logs the event. |
| Unrecognised command | Rejected; client is informed the command is unsupported.                             |
| Oversized message    | Rejected before being broadcast or logged.                                           |

### 4.3 Session Timeout

The server records a last-activity timestamp for every connected client and periodically checks it against a configured inactivity threshold. If a client exceeds this threshold without sending any message, the server closes that client's session, logs a timeout event, and notifies the remaining clients that the user has disconnected due to inactivity.

### 4.4 Secure Logging

All security-relevant events are appended to security\_log.txt with a timestamp, event type, and username, but never with password data. Example categories of log entries include:

- LOGIN\_SUCCESS – user, timestamp
- LOGIN\_FAILED – user, timestamp, attempt count
- ACCOUNT\_LOCKED – user, timestamp, lockout duration
- DUPLICATE\_LOGIN\_ATTEMPT – user, timestamp
- SESSION\_TIMEOUT – user, timestamp
- LOGOUT – user, timestamp

## 5. Testing

---

The application was tested on the Mininet single,5 topology with one server host and four client hosts. The following test cases were executed to validate each security feature:

| Test Case                                            | Expected Result                                 |
|------------------------------------------------------|-------------------------------------------------|
| Login with correct credentials                       | Access granted; session created                 |
| Login with incorrect password                        | Access denied; failed-attempt count incremented |
| Five consecutive failed logins                       | Account temporarily locked                      |
| Duplicate login with same username from another host | Second login rejected                           |
| Empty username / empty password                      | Login request rejected                          |

|                                               |                                             |
|-----------------------------------------------|---------------------------------------------|
| Oversized chat message                        | Message rejected before broadcast           |
| Unsupported command                           | Command rejected with error response        |
| Client inactivity beyond timeout period       | Session automatically closed                |
| Manual logout via Disconnect / /logout        | Session closed and removed from online list |
| Inspection of users.json and security_log.txt | No plaintext passwords present              |

Screenshots of each test scenario — including successful login, invalid login, account lockout, duplicate login rejection, session timeout, broadcast and private messaging, the online users list, and the contents of security\_log.txt — are provided in the screenshots/ directory accompanying this report.

## 5.1 Test Screenshots

The following screenshots, captured during testing, illustrate the application's response to each of the key security scenarios listed in the test table above.

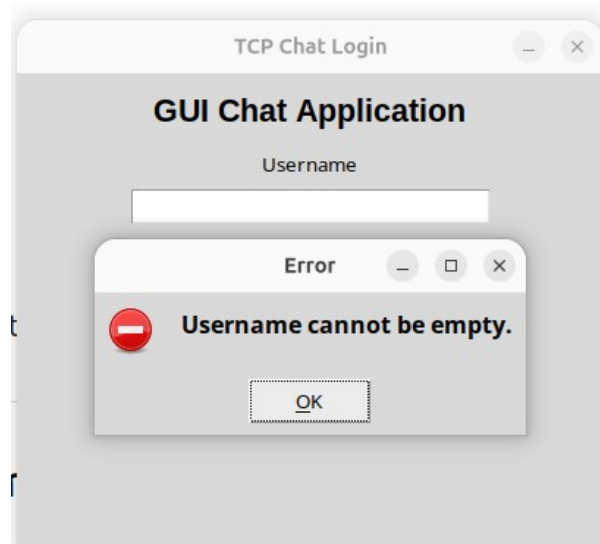


Fig. 5.1 — Empty username rejected during login (input validation).

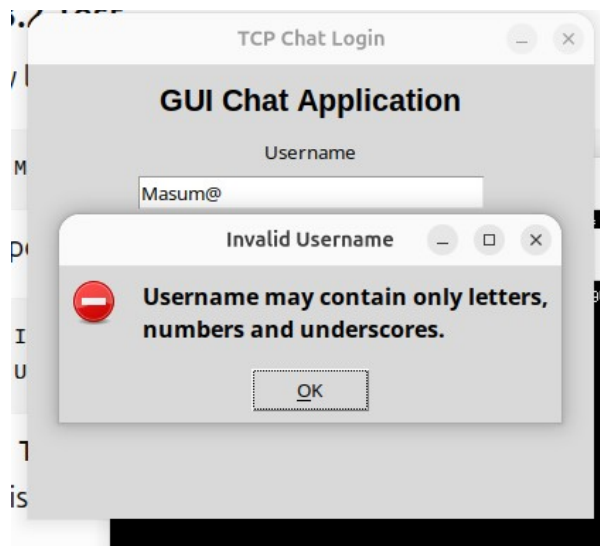


Fig. 5.2 — Invalid username format rejected; only letters, numbers and underscores are permitted.

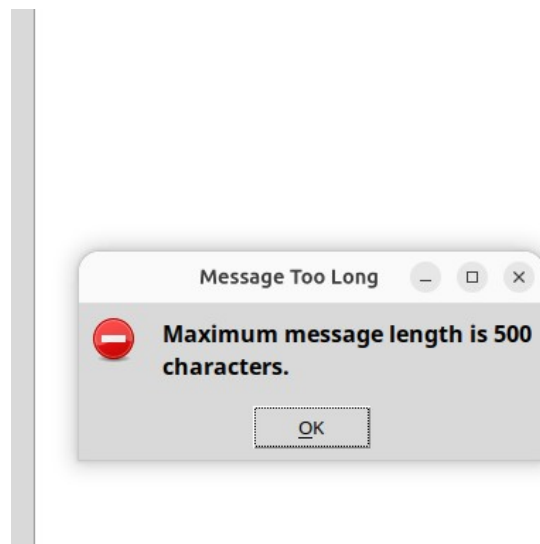


Fig. 5.3 — Oversized chat message rejected with a maximum length notification.

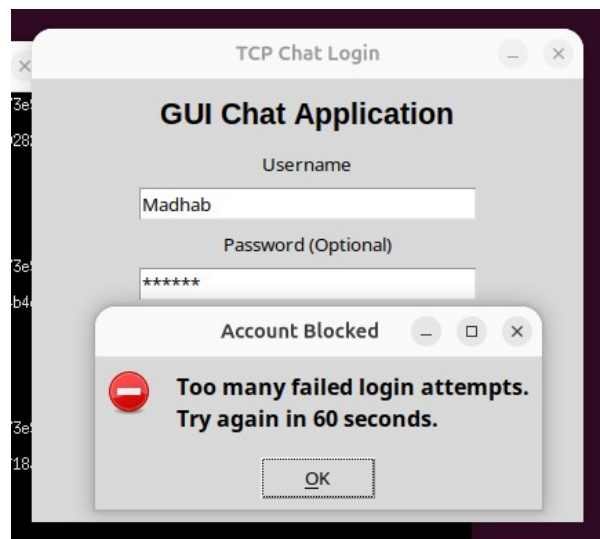


Fig. 5.4 — Account temporarily blocked after five consecutive failed login attempts.

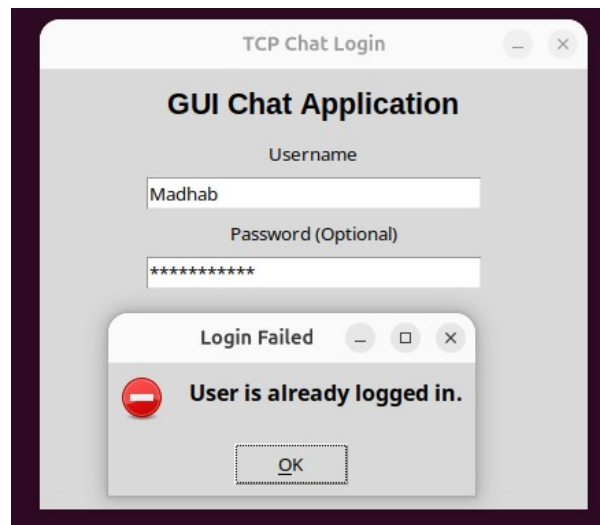


Fig. 5.5 — Duplicate login attempt rejected for a username that is already active.

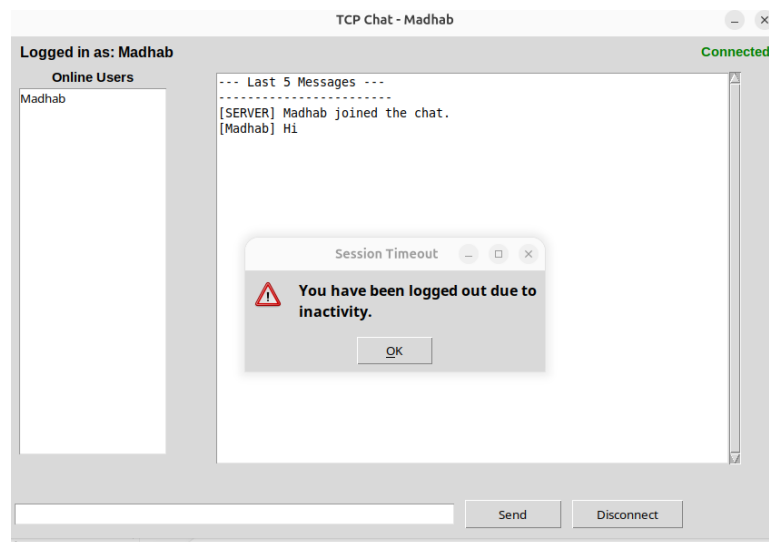


Fig. 5.6 — Client automatically logged out by the server after a period of inactivity.

## 6. Wireshark Verification

Network traffic generated by the application was captured using Wireshark with the display filter `tcp.port == 5000` to isolate traffic belonging to the chat application. The following aspects of communication were verified:

- TCP three-way handshake (SYN, SYN-ACK, ACK) establishing the connection between client and server.
- Login request and response packets, confirming that authentication data is transmitted only over the established TCP session.
- Failed login attempt packets, confirming that repeated failures are visible as distinct exchanges without exposing plaintext credentials in the packet payload.
- Broadcast and private message packets exchanged after successful authentication.
- Session timeout and logout packets, followed by TCP connection termination (FIN/ACK).

Inspection of the captured packets confirmed that passwords are transmitted as SHA-256 hash values rather than plaintext, consistent with the secure password handling implemented on both the client and server. Corresponding Wireshark screenshots are included in the screenshots/wireshark/ directory.

## 7. Conclusion

---

This assignment successfully extended the multi-client TCP chat application from Assignment 6 with practical, application-level security mechanisms. The implementation demonstrates secure password storage using SHA-256 hashing, robust authentication with duplicate-login and brute-force protection, thorough input validation, automatic session timeout, and comprehensive security event logging that avoids exposing sensitive data. Verification using Wireshark confirmed that authentication and communication behave as intended at the network level, with no plaintext credentials observed on the wire or in stored logs. Overall, the project reinforces core concepts of authentication, authorization, and secure session handling within a real client-server networking environment.